

Requirement Understanding

Case Study: Community Library Membership & Book Lending System

Case Study Requirement

A community library wants a console-based application to manage:

- Members
- Books
- Book categories
- Book copies
- BorrowingwordMem
- Returns
- Fine calculation
- Membership limits

The system should use PostgreSQL as the database and EF Core for database access.

Core Functional Requirements

1. Member Management

The system should allow:

- Add a new member
- View all members
- Search member by phone number or email
- Update membership status
- Deactivate a member

Each member should have a membership type such as:

- Basic
- Premium
- Student

2. Book Management

The system should allow:

- Add a new book
 - Add multiple copies of a book
 - View available books
 - Search books by title, author, or category
 - Mark a book copy as damaged or unavailable
-

3. Borrowing Rules

The system must apply business logic before allowing a member to borrow a book.

Business Logic Criteria

Membership Type	Maximum Active Borrowings	Maximum Borrow Days
Basic	2 books	7 days
Student	3 books	10 days
Premium	5 books	15 days

A member cannot borrow a book if:

- Their membership is inactive
 - They already reached the borrowing limit
 - The book copy is not available
 - They have unpaid fines above ₹500
 - They already borrowed the same book and have not returned it
-

4. Return Rules

When a book is returned:

- The system should calculate late return fine

- Fine amount = ₹10 per delayed day
 - The returned book copy should become available again
 - Borrowing status should change to returned
 - Return date should be saved
-

5. Fine Management

The system should allow:

- View pending fines of a member
 - Pay fine
 - View fine history
 - Prevent borrowing if unpaid fine is above ₹500
-

Stored Procedure / PostgreSQL Function Requirement

Participants must create at least one PostgreSQL function and call it from EF Core.

Suggested functions:

1. `calculate_member_fine(member_id)`
 - o Returns total unpaid fine for a member
 2. `get_available_books_by_category(category_id)`
 - o Returns available books under a category
 3. `get_member_borrowing_summary(member_id)`
 - o Returns active borrowed books, returned books, and total fine
-

Transaction Requirement

Participants must use EF Core transaction for the borrowing process.

Borrowing a book should happen as one transaction:

1. Validate member

2. Validate unpaid fine
3. Check active borrowing count
4. Check book copy availability
5. Create borrowing record
6. Update book copy status as borrowed
7. Commit transaction

If any step fails, rollback the transaction.

Suggested Console Menu

1. Member Management
 2. Book Management
 3. Borrow Book
 4. Return Book
 5. Fine Management
 6. Reports
 7. Exit
-

Reports

Add report options such as:

- Books currently borrowed
 - Overdue books
 - Members with pending fines
 - Most borrowed books
 - Available books by category
 - Member borrowing history
-